README | No Optimizado

Simulación de trayectorias circulares y formación de cuatro agentes.

Archivos

Archivo	Función
pid_circular_normal.m	PID lineal y PI angular; objetivo por tolerancia.
pid_circular_exponencial.m	Velocidad exponencial con v0 = Ra/2.
pid_circular_purepursuit(1).m	Velocidad exponencial con v0 = 2 m/s.
unicycle_lqrlqi.m	LQI; referencia circular actualizada por tiempo.
Pruebas_1.m	Pruebas de trayectorias y partículas, sin realimentación.

Requisitos y ejecución

MATLAB con interfaz gráfica. `unicycle_lqrlqi.m` requiere la función `lqr` de Control System Toolbox. Las dos primeras secciones de `Pruebas_1.m` requieren `waypointTrajectory`. Los demás scripts usan operaciones numéricas y gráficas básicas.

Desde la carpeta de los archivos, ejecutar el script deseado:

```
run('pid_circular_normal.m')
run('pid_circular_exponencial.m')
run('pid_circular_purepursuit(1).m')
run('unicycle_lqrlqi.m')
run('Pruebas_1.m')
```

Los scripts de control empiezan con `clear all`. En `Pruebas_1.m`, usar Run Section para ejecutar cada prueba por separado; varias secciones borran variables y cierran figuras.

Modelo común de los cuatro scripts de control

Estado: `xi = [x; y; theta]`. Entrada: `u = [v; omega]`. Unidades: metros, segundos y radianes. Los estados se integran con RK4.

```
dx/dt     = v*cos(theta)
dy/dt     = v*sin(theta)
dtheta/dt = omega
```

Resultados

`XI`, `XI1`, `XI2`, `XI3` guardan los estados; `U`, `U1`, `U2`, `U3`, las entradas. `traj` contiene el círculo y `G`, los objetivos del líder. Se animan los cuatro robots; las gráficas temporales muestran solo al líder y a los dos primeros seguidores. No hay exportación automática.

Control PID y ley exponencial

Configuración compartida

Parámetro	Valor
NV / dt / tf	3 vueltas generadas / 0.01 s / 24 s
Ra / centro	5 m / (0, 0)
w	NV*2*pi/tf = pi/4 rad/s
d1, d2, d3	0.5 m cada uno
lookahead_samples / ePtol	20 muestras / 0.5 m
kpO / kiO / kdO	10 / 0.0001 / 0

El líder inicia en `[5; 0; pi/2]`; los seguidores, en x = 4.5, 4 y 3.5 m, con y = 0 y theta = pi/2. El círculo se genera con:

```
xg(t) = xc + Ra*cos(w*t)
yg(t) = yc + Ra*sin(w*t)
```

Cuando la distancia al objetivo cumple `eP <= ePtol`, el índice del líder avanza 20 muestras y se limita al último punto. Cada seguidor apunta detrás del agente precedente:

```
ref = p_prev - d*[cos(theta_prev); sin(theta_prev)];
```

Aquí `p_prev` y `theta_prev` representan la posición y orientación actuales del agente precedente. El avance depende del error; `NV` no garantiza las vueltas recorridas. Es persecución de puntos por umbral, sin una ley geométrica de curvatura de Pure Pursuit.

1. pid_circular_normal.m

La velocidad lineal usa el error de distancia `eP`. Ganancias: `kpP = 3`, `kiP = 0.0001`, `kdP = 0.5`.

```
EP = EP + eP;
v  = kpP*eP + kiP*EP + kdP*(eP - eP_prev);
```

2. pid_circular_exponencial.m

Sustituye el PID lineal por la ley siguiente, con `v0 = Ra/2 = 2.5 m/s` y `alpha = 10`. Para `eP <= 1e-6`, ordena v = 0.

```
kP = v0*(1 - exp(-alpha*eP^2))/eP;
v  = kP*eP;    % equivale a v0*(1 - exp(-alpha*eP^2))
```

Si `Ra <= 50`: salto de 20 muestras y tolerancia de 0.5 m. Si `Ra > 50`: 35 muestras y 1 m.

3. pid_circular_purepursuit.m

Usa la misma ley exponencial, con `v0 = 2 m/s`, `alpha = 10` y anticipación fija de 20 muestras. Solo `li0` cambia el objetivo; `li1`, `li2` y `li3` no se utilizan.

Regulación angular de las tres variantes

`eO` es el error angular normalizado con `atan2(sin(error), cos(error))`. Como kdO = 0, el regulador configurado es PI:

```
EO    = EO + eO;
omega = kpO*eO + kiO*EO;
```

Las sumas integrales y diferencias derivativas de estas variantes no se escalan con dt. Cambiar el muestreo requiere revisar las ganancias.

4. unicycle_lqrlqi.m

Diseña un LQI mediante LQR sobre un sistema aumentado con integradores. El líder usa `traj(:, k+1)` en cada paso; los seguidores persiguen puntos detrás del agente anterior.

Parámetros

Parámetro	Valor
NV / dt / tf	2 vueltas de referencia / 0.01 s / 16 s
Ra / centro / w	23 m / (0, 0) / pi/4 rad/s
ell	0.1 m
d / d2 / d3	0.5 / 1 / 1.5 m, respecto al agente precedente
QQ / R	diag([1 1 500 500]) / 10*eye(2)

Los estados iniciales son `[0;0;0]`, `[-0.5;0;0]`, `[-1;0;0]` y `[-1.5;0;0]`. El círculo empieza en `[23;0]`, separado de la posición inicial del líder.

Diseño y ley de control

```
A  = zeros(2);
B  = eye(2);
Cr = eye(2);
Dr = zeros(2);
AA = [A, zeros(2); Cr, zeros(2)];
BB = [B; Dr];
QQ = diag([1 1 500 500]);
R  = 10*eye(2);
Klqi = lqr(AA, BB, QQ, R);
```

En cada iteración, `x = xi(1:2)` y el error se integra incluyendo dt:

```
sigma = sigma + (Cr*x - ref)*dt;
mu    = -Klqi*[x; sigma];
u     = finv(xi, mu);
```

La función `finv` convierte la entrada virtual `mu = [mu_x; mu_y]` en velocidades:

```
v     = cos(theta)*mu_x + sin(theta)*mu_y;
omega = (-sin(theta)*mu_x + cos(theta)*mu_y)/ell;
```

Cada seguidor añade `0.001*(ref - x)` a su entrada virtual antes de aplicar `finv`. La ganancia `Klqr` también se calcula, pero no se usa en el control.

Detalle de implementación

La transformación corresponde al punto adelantado `p = [x;y] + ell*[cos(theta);sin(theta)]`, mientras la realimentación utiliza el centro `xi(1:2)`. Esa diferencia debe revisarse antes de asumir una linealización exacta de la salida controlada. El comentario sobre un radio máximo de 22 indica el máximo de agentes posibles; el archivo configura 23 para mostrar la falla que sucede al aumentar el radio máximo.

Límites comunes de los scripts de control

No incluyen prevención de colisiones ni anti-windup. Las variantes PID no limitan la velocidad angular; el PID normal y el LQI tampoco saturan la velocidad lineal. Las distancias traseras son referencias, no restricciones exactas de separación.

5. Pruebas_1.m

Reúne siete pruebas independientes. Las posiciones se generan directamente; no se simula un controlador de seguimiento. Hay dos secciones tituladas Pru 4.

Pruebas disponibles

Sección	Contenido y parámetros
Pru 1	Waypoints en XY: [0,0,0], [1,1,0], [0,2,0].
Pru 2	Waypoints en XZ: [0,0,0], [1,0,1], [0,0,2].
Pru 3	Una partícula: R = 5, omega = 2; 400 muestras entre 0 y 4*pi.
Pru 4, primera	Dos partículas simultáneas: R = 5 y omega = 1; 150 muestras entre 0 y 4*pi.
Pru 4, segunda	Dos partículas; 200 muestras entre 0 y 4*pi. Retraso de 25 muestras para la segunda.
Pru 5	Cuatro partículas: R = 5, omega = 1; 500 muestras entre 0 y 10*pi. Retrasos: 25, 50 y 75 muestras.
Pru 6	Configuración de Pru 5 con longitud_estela = 25.

Generación y animación

Pru 1 y Pru 2 usan `waypointTrajectory`, `TimeOfArrival = [0;1;2]` y `SampleRate = 10`. Guardan posiciones en `truePosition` y dibujan la trayectoria con `plot3`. Las otras pruebas usan:

```
x(t)  = R*cos(omega*t)
y(t)  = R*sin(omega*t)
vx(t) = -R*omega*sin(omega*t)
vy(t) =  R*omega*cos(omega*t)
```

Las flechas de `quiver` representan velocidad tangencial, aunque el código las llama gradientes. `pause` modifica la reproducción, no el tiempo usado en las ecuaciones.

Detalles a tener en cuenta

En Pru 3 se generan cuatro vueltas, porque omega*(4*pi) = 8*pi. Las dos partículas de la prueba simultánea tienen iguales parámetros y fase: se superponen.

Una partícula diferida aparece cuando `k > delay_frames`, usando `idx = k - delay_frames`. El ciclo no se extiende para compensar el retraso; las partículas diferidas recorren menos muestras.

En Pru 6, el intervalo `max(1,idx-longitud_estela):idx` contiene hasta 26 puntos. En las pruebas de cuatro partículas, los límites gráficos usan solo `max(R1,R4)`; deben revisarse si se modifican R2 o R3.

Notas de uso

Los resultados quedan en el Workspace y en las figuras. Ningún archivo exporta resultados automáticamente. Si cambia el centro del círculo en las variantes PID, también deben ajustarse sus posiciones iniciales. En todos los controladores, cambiar solo NV conserva w porque tf = 8*NV.